استفاده می‌کنیم. در حقیقت، زمانبندی نخ‌های هسته را در سیستم‌های سولاریس و ویندوز و وظایف را با زمانبند لینوکس مطرح می‌کنیم.

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[]) {
        int i, policy;
        pthread_t tid[NUM THREADS];
        pthread_attr_t attr;
        /* get the default attributes */
        pthread_attr_init(&attr);
        /* get the current scheduling policy */
        if (pthread_attr_getschedpolicy(&attr, &policy) != 0)
                fprintf(stderr, "Unable to get policy.\n");
        else {
                if (policy == SCHED_OTHER)
                        printf("SCHED_OTHER\n");
                else if (policy == SCHED_RR)
                        printf("SCHED_RR\n");
                else if (policy == SCHED_FIFO)
                        printf("SCHED_FIFO\n");
        }
        /* set the scheduling policy - FIFO, RR, or OTHER */
        if (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) != 0)
                fprintf(stderr, "Unable to set policy.\n");
        /* create the threads */
        for (i = 0; i < NUM_THREADS; i++)
                pthread_create(&tid[i],&attr,runner,NULL);
        /* now join on each thread */
        for (i = 0; i < NUM_THREADS; i++) pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
        /* do some work ... */
        pthread_exit(0);
}
```

شکل۵-۲۵ API زمانبندی بلادرنگ پوزیکس.

۵-۷-۱ مثال: زمانبندی لینوکس

زمانبندی پردازه در لینوکس پیشینه‌ی جالبی داشته است. پیش از نسخه‌ی 2.5، هسته‌ی لینوکس گونه‌ای از الگوریتم زمانبندی سابق یونیکس را اجرا می‌کرد. با این وجود، چون طراحی این

الگوریتم ذهنیتی از سیستم‌های SMP داشت، به قدر کافی سیستم‌های با پردازنده‌های متعدد را پشتیبانی نمی‌کند. به علاوه، در سیستم‌هایی که تعداد زیادی پردازه‌ی آماده برای اجرا دارند به کارایی ضعیف منجر می‌شود. با نسخه‌ی 2.5 هسته، زمانبند با لحاظ یک الگوریتم زمانبندی– معروف به O(1)–به دقت ترمیم یافت که فارغ از تعداد وظیفه‌های سیستم، با زمان ثابتی اجرا شد. زمانبند O(1)، پشتیبانی بیشتری نیز برای SMP، اعم از خویشاوندی پردازنده و تعادل بار را میان پردازنده‌ها فراهم می‌کند. با این وجود، در عمل، هرچند که زمانبند O(1) کارایی ممتازی روی سیستم‌های SMP داشت، برای پردازه‌های محاوره‌ای که در بسیاری از سیستم‌های کامپیوتری رومیزی رایج هستند به زمان‌های پاسخ ضعیفی منجر شد. در طی توسعه‌ی هسته‌ی 2.6، باز هم زمانبند دچار تغییراتی شد؛ و در نسخه‌ی 2.6.23 هسته، *زمانبند کاملاً منصف*[1] (CFS) به عنوان الگوریتم پیش‌فرض زمانبندی لینوکس انتخاب شد.

زمانبندی در سیستم لینوکس مبتنی بر **رده‌های زمانبندی** است. به هر رده یک الویت خاص نسبت داده می‌شود. هسته با استفاده از رده‌های زمانبندی متفاوت می‌تواند مطابق نیازهای سیستم و پردازه‌هایش، الگوریتم‌های زمانبندی متفاوتی را تحت پوشش قرار بدهد. برای مثال، معیارهای زمانبندی برای یک کارگزار لینوکس ممکن است متفاوت با آنهایی باشد که برای یک دستگاه موبایل که لینوکس را اجرا می‌کند، انتخاب می‌شوند. زمانبند به منظور تصمیم‌گیری در رابطه با وظیفه‌ی اجرایی بعدی، وظیفه‌ی با الویت بالاترین را از رده‌ی زمانبندی با الویت بالاترین انتخاب می‌کند. هسته‌های استاندارد لینوکس، دو رده‌ی زمانبندی را پیاده‌سازی می‌کنند که عبارتند از: (۱) رده‌ی زمانبندی پیش‌فرض با استفاده از الگوریتم زمانبندی CFS و (۲) رده‌ی زمانبندی بلادرنگ. هر یک از این رده‌ها را در اینجا مورد بحث قرار می‌دهیم. البته، رده‌های زمانبندی جدیدی هم می‌توان اضافه کرد.

زمانبند CFS به جای استفاده از قواعد محضی که یک مقدار الویت نسبی را به طول یک کمیت زمانی مرتبط می‌کنند، نسبتی از زمان پردازشی پردازنده را در اختیار هر یک از وظیفه‌ها قرار می‌دهد. این نسبت مطابق **مقدار عالی** که برای هر وظیفه انتخاب می‌شود، محاسبه می‌گردد. مقادیر عالی در بازه‌ی ۲۰- تا ۱۹+ قرار دارند که یک مقدار عالی عددی پایین‌تر نشان از الویت نسبی بالاتر است. وظایف با مقادیر عالی پایین‌تر، نسبت بالاتری از زمان پردازشی پردازنده را به نسبت وظایف با مقادیر عالی بالاتر دریافت می‌کنند. مقدار عالی پیش‌فرض ۰ است. (اصطلاح ***عالی*** از این ایده بر می‌آید که اگر یک وظیفه مقدار عالی خود را بر فرض، از ۰ تا ۱۰+ افزایش بدهد، این کار، یعنی کاهش الویت نسبی وظیفه برای وظایف دیگر سیستم عالی است. به‌عبارت دیگر، پردازه‌های عالی دیر به‌اتمام می‌رسند!) CFS از مقادیر گسسته‌ی برش‌های زمانی استفاده نمی‌کند و در عوض، یک **دیرکرد هدف**[2] مشخص می‌کند که یک فاصله‌ی زمانی است که هر وظیفه‌ی قابل اجرا بایستی حداقل یک مرتبه اجرا در آن داشته باشد. نسبت‌های زمان پردازنده از روی مقدار دیرکرد

1. Completely Fair Scheduler
2. Targeted latency

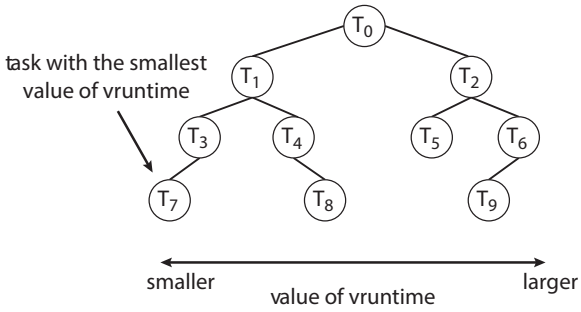
هدف تخصیص می‌یابند. دیرکرد هدف علاوه بر داشتن مقادیر پیش‌فرض و کمینه می‌تواند در صورتی که تعداد وظایف فعال سیستم فراتر از یک آستانه‌ی معین رشد پیدا کند، افزایش یابد.

زمانبند CFS الویت‌ها را به صورت مستقیم انتخاب نمی‌کند. در عوض، با نگه داشتن **زمان اجرای مجازی** هر وظیفه با استفاده از متغیر vruntime وظیفه، مدت زمانی را که وظیفه اجرا داشته است ثبت می‌کند. زمان اجرای مجازی مطابق الویت یک وظیفه با ضریب کاهشی مرتبط است: وظایف با الویت پایین‌تر نسبت به وظایف با الویت بالاتر دارای ضرایب کاهش بالاتری هستند. برای وظایف با الویت نرمال (مقادیر عالی ۰)، زمان اجرای مجازی با زمان اجرای فیزیکی واقعی یکی است. بنابراین، اگر یک وظیفه با الویت پیش‌فرض به مدت ۲۰۰ میلی‌ثانیه اجرا شود، vruntime آن نیز ۲۰۰ میلی‌ثانیه خواهد بود. با این وجود، اگر وظیفه‌ای با الویت پایین‌تر به مدت ۲۰۰ میلی‌ثانیه اجرا شود، vruntime آن بیشتر از ۲۰۰ میلی‌ثانیه خواهد شد. به طور مشابه، اگر وظیفه‌ای با الویت بالاتر به مدت ۲۰۰ میلی‌ثانیه اجرا شود، vruntime آن کمتر از ۲۰۰ میلی‌ثانیه خواهد بود. زمانبند برای تصمیم‌گیری در رابطه با وظیفه‌ی اجرایی بعدی، به سادگی وظیفه‌ای را انتخاب می‌کند که کوچک‌ترین مقدار vruntime را داراست. به علاوه، یک وظیفه‌ی با الویت بالاتر که برای اجرا آمادگی پیدا می‌کند، می‌تواند توسط یک وظیفه‌ی با الویت پایین‌تر پس گرفته شود.

بیایید زمانبند CFS را در عمل ببینیم: فرض کنید که دو وظیفه دارای مقادیر عالی یکسان هستند. یک وظیفه پابند I/O و دیگری پابند پردازنده است. نوعاً، وظیفه‌ی پابند I/O پیش از این که مسدود شود یا I/O دیگری بخواهد تنها برای یک مدت کوتاهی اجرا خواهد داشت و وظیفه‌ی پابند پردازنده موقعی که فرصت می‌یابد روی یک پردازنده اجرا داشته باشد، دوره‌ی زمانی اجرایی خود را کاملاً مصرف خواهد کرد. بنابراین، مقدار vruntime در نهایت برای وظیفه‌ی پابند I/O نسبت به وظیفه‌ی پابند پردازنده کمتر خواهد شد که این کار، الویت بالاتری را به وظیفه‌ی پابند I/O در مقایسه با وظیفه‌ی پابند پردازنده خواهد داد. در اینجا، اگر موقعی که وظیفه‌ی پابند I/O برای اجرا آمادگی پیدا می‌کند (برای مثال، موقعی که I/O مورد انتظار وظیفه فراهم شود)، وظیفه‌ی پابند پردازنده در حال اجرا باشد، وظیفه‌ی پابند I/O وظیفه‌ی پابند پردازنده را پس خواهد زد.

کارایی CFS

زمانبند CFS لینوکس، یک الگوریتم کارآمد برای انتخاب وظیفه‌ی اجرایی بعدی فراهم می‌کند. هر وظیفه‌ی قابل اجرا در یک درخت قرمز–سیاه□درخت جستجوی دودویی متوازن که کلید آن مطابق مقدار vruntime است□گذاشته می‌شود. این درخت در زیر نشان داده می‌شود:



موقعی که یک وظیفه برای اجرا آماده است، به درخت افزوده می‌شود. اگر یک وظیفه در درخت آمادگی اجرا نداشته باشد (برای مثال، اگر در حال انتظار برای I/O مسدود شود)، حذف می‌گردد. به طور کلی، به وظایفی که زمان پردازشی کمتری (مقادیر vruntime کوچک‌تر) داده می‌شود به سمت چپ درخت روانه می‌شوند و وظایفی که زمان پردازشی بیشتری می‌گیرند در سمت راست قرار داده می‌شوند. بر اساس ویژگی‌های درخت جستجوی دودویی، چپ‌ترین گره، دارای کوچک‌ترین مقدار کلید است که برای زمانبند CFS این معنی را دارد که همان وظیفه‌ی با الویت بالاترین است. چون درخت قرمز-سیاه متوازن است، پیمایش آن در یافتن چپ‌ترین گره به عملیاتی از مرتبه‌ی $O(\lg N)$ (که N تعداد گره‌های درخت است) نیاز خواهد داشت. با این وجود، به دلایل کارآمدی، زمانبند لینوکس، این مقدار را در متغیر rb_leftmost کش می‌سازد و لذا تعیین وظیفه‌ی اجرایی بعدی، تنها مستلزم بازیابی مقدار کش شده است.

لینوکس، زمانبندی بلادرنگ را نیز با استفاده از استاندارد پوزیکس آن‌گونه که در بخش ۵-۶ بیان شد، پیاده‌سازی می‌کند. هر وظیفه‌ای که با استفاده از سیاست بلادرنگ SCHED_FIFO و یا SCHED_RR زمانبندی شود، با الویت بالاتری از وظایف نرمال (غیربلادرنگ) اجرا می‌شود. لینوکس از دو بازه‌ی الویت مجزا، اولی برای وظایف بلادرنگ و دومی برای وظایف نرمال استفاده می‌کند. برای وظایف بلادرنگ، الویت‌های ایستایی از بازه‌ی ۰ تا ۹۹ انتخاب می‌شود و برای وظایف نرمال، الویت‌هایی از ۱۰۰ تا ۱۳۹ انتخاب می‌شود. این دو بازه به یک طرح الویت سراسری نگاشته می‌شوند که در آن، مقادیر عددی پایین‌تر نشان از الویت‌های نسبی بالاتر است. الویت وظایف نرمال مطابق مقادیر عالی آنها انتخاب می‌شود که مقدار ۲۰- به الویت ۱۰۰ و مقدار عالی ۱۹+ به ۱۳۹ نگاشته می‌شود. این طرح در شکل ۵-۲۶ نشان داده می‌شود.

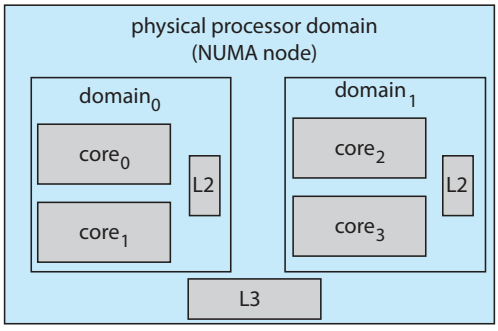


شکل ۵-۲۶ الویت‌های زمانبندی در سیستم لینوکس.

زمان‌بند CFS با استفاده از یک تکنیک پیچیده‌ای که بار را مابین هسته‌های پردازشی برابر می‌کند از تعادل بار پشتیبانی می‌کند، ضمن این که آگاه به NUMA نیز بوده و مهاجرت نخ‌های را نیز به‌حداقل می‌رساند. CFS بار هر کدام از نخ‌ها را به‌صورت ترکیبی از الویت نخ و نرخ متوسط

بهره‌وری پردازنده‌اش تعریف می‌کند. بنابراین، نخی که الویت بالایی دارد، باز هم بیشتر عطش I/O داشته و به استفاده‌ی پردازنده کمتر نیاز دارد، بار کلی پایینی دارد، شبیه به بار یک نخ با الویت پایینی که بهره‌وروی پردازنده‌ی بالایی دارد. با استفاده از این معیار، بار یک صف برابر مجموع بارهای تمامی نخ‌های صف می‌باشد و موازنه به‌سادگی تضمین این است که تمامی صف‌ها تقریباً بار یکسانی دارند.

با این وجود، همان‌گونه که در بخش ۵-۵-۴ تأکید شد، مهاجرت یک نخ ممکن است نتیجه‌اش جریمه‌ی دسترسی حافظه باشد، به‌واسطه‌ی بی‌اعتبارکردن محتویات کَش، و یا، در سیستم‌های NUMA، تحمل زمان‌های دسترسی طولانی‌تر حافظه. برای رفع این مسئله، لینوکس یک سیستم سلسله‌مراتبی از دامنه‌های زمان‌بندی مشخص می‌کند. **دامنه‌ی زمان‌بندی** مجموعه‌ای از هسته‌های پردازنده است که می‌توانند در مقابل یکدیگر موازنه شوند. این اصل در شکل ۵-۲۷ نمایش داده می‌شود. هسته‌ها در هر دامنه‌ی زمان‌بندی مطابق چگونگی به‌اشتراک‌گذاری منابع سیستم آن‌ها گروه‌بندی می‌شوند. برای مثال، هرچند که هر هسته‌ی نشان داده شده در شکل ۵-۲۷ ممکن است کَش سطح ۱ (L1) خاص خودش را داشته باشد، جفت‌هسته‌ها کَش سطح ۲ (L2) را به‌اشتراک می‌گذارند و لذا به‌شکل $domain_0$ و $domain_1$ جداگانه سازمان‌دهی می‌شوند. همین‌طور، این دو دامنه ممکن است کَش سطح ۳ (L3) را به‌اشتراک بگیرند، و از این رو، به‌شکل یک دامنه‌ی سطح پردازنده (که به **گره NUMA** نیز معروف است) سازمان‌دهی شوند. در یک سیستم NUMA با برداشتن یک گام بیشتر در این راستا، یک دامنه‌ی سطح سیستم بزرگ‌تری گره‌های NUMA سطح پردازنده‌ی جداگانه را ترکیب خواهد نمود.



شکل ۵-۲۷ تعادل بار آگاه به NUMA با زمان‌بند CFS لینوکس.

راهبرد کلی پشت سر CFS، موازنه‌ی بارها در میان دامنه‌ها با شروع از پایین‌ترین سطح سلسله‌مراتب است. با استفاده از شکل ۵-۲۷ به‌عنوان یک مثال، یک نخ در ابتدا تنها میان هسته‌های روی دامنه‌ای یکسان مهاجرت خواهد داشت (یعنی، در میان $domain_0$ یا $domain_1$). تعادل بار در سطح بعدی میان $domain_0$ و $domain_1$ صورت خواهد گرفت. CFS از مهاجرت نخ‌ها میان گره‌های مجزای NUMA جلوگیری خواهد کرد اگر یک نخ دورتر از حافظه‌ی محلیش جابجا شود، و چنین مهاجرتی تنها در صورتی که عدم تعادل بار جدی وجود داشته باشد اتفاق خواهد افتاد.